

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-206Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №3

Задача: Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь.

Вариант структуры данных: PATRICIA.

Вариант ключа: регистронезависимая последовательность букв английского алфавита длиной не более 256 символов.

Вариант значения: числа от 0 до $2^{64} - 1$.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ **word 34** — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- **word** — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

! Save /path/to/file — сохранить словарь в бинарном компактном представлении на диск в файл, указанный параметром команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки (см. ниже).

! Load /path/to/file — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле.

Для всех операций, в случае возникновения системной ошибки (нехватка памяти, отсутствие прав записи и т.п.), программа должна вывести строку, начинающуюся с «ERROR:» и описывающую на английском языке возникшую ошибку.

Также необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

1 Описание

Требуется реализовать структуру данных PATRICIA. Основная ее идея заключается в том, чтобы при обходе вершин сравнивать не строки целиком, а один нужный бит. Пользуясь информацией из [1], мы понимаем, что сложность поиска будет равна $\Theta(h)$, где h - высота дерева, так как мы сравниваем только один бит в нашей строке, и далее идем в нужного сына. Вставка в данной структуре данных работает за $\Theta(2h) = \Theta(h)$, потому что для данного алгоритма нам нужно сначала найти «похожий» элемент, а потом место для вставки. Для удаления сложность так же будет $\Theta(h)$, для алгоритма надо четыре раза применить поиск в дереве. Высота h в дереве находится в промежутке от $\log(n)$ до n .

2 Исходный код

Для каждого узла в дереве у меня хранится динамический массив `char`, номер бита `int`, данные в `unsigned long long` и указатели на двоих сыновей. В строке мы будем хранить изначальный ключ, а не его битовое представление. Таким образом мы сократим затраты памяти. Во время алгоритмов поиска, вставки и удаления на каждой итерации будем осуществлять побитовый сдвиг текущего ключа, чтобы узнать нужный бит.

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 |
5 | typedef struct Patricia {
6 |     unsigned long long data;
7 |     int bit;
8 |     char* word;
9 |
10 |    struct Patricia* left;
11 |    struct Patricia* right;
12 | } Patricia;
13 | }
```

patricia.c	
Patricia* createPatricia(unsigned long long value, char* str);	Создание дерева
Patricia* search(Patricia* node, char* str, int lastbit);	Поиск в дереве по ключу str
Patricia searchNodeForInsert(Patricia* node, Patricia searched, int searched_bit, Patricia parent, int lastbit);	Вспомогательная функция для вставки в дерево
void insertToPatricia(Patricia* node, char str, unsigned long long value, int ok_or_not);	Алгоритм вставки в дерево
Patricia* searchBackLinkToNode(Patricia* node, Patricia searched, Patricia parent, int lastbit);	Вспомогательная функция для удаления из дерева, ищет обратную ссылку на переданный узел
Patricia searchDirectLinkToNode(Patricia* node, Patricia searched, Patricia parent, int lastbit);	Вспомогательная функция для удаления из дерева, ищет прямую ссылку на переданный узел
void removeNode(Patricia root, char str);	Алгоритм удаления из дерева
void printOneNode(Patricia node);	Вспомогательная функция для вывода дерева

void printPatriciaInAddress(Patricia × root, int last_bit);	Функция вывода дерева со всей информацией о каждом узле
void deletePatricia(Patricia node, int last_bit);	Функция удаления дерева

Для реализации ввода я использовал цикл while. В каждой итерации цикла я читаю введенную строку, и проверяю, подходит ли ее формат под чтение, загрузку, удаление узла или загрузку и сохранение в файл. Чтобы записать дерево в файл, я сначала открываю файл на запись в бинарном виде, прохожусь по каждому узлу в дереве и записываю ключ и значение с помощью команды fwrite(). Для чтения из файла я также открываю файл на чтение в бинарном виде и читаю по очереди размер ключа, ключ и значение с помощью команды fread().

laba2.c	
char toLower(char ch)	Возвращает маленький регистр буквы
void toLowercase(char str[500])	Делает каждый символ строки маленького регистра
void writePatriciaToFile(FILE file, Patricia node, int last_bit)	Запись дерева в файл
Patricia readPatriciaFromFile(FILE file, int check_correct)	Чтение дерева из файла

3 Консоль

```
dmitrykochev$ cat test1.txt
+ a 1
+ aa 2
+ aaaaa 5
+ aaaaaa 6
+ aaaaaaa 7
+ aaaaaaaaa 10
+ bbccsdeqwww 11
aaaa
aaaaaaa
aaaaaaaa
aaaaaa
-aaaaaa
-aaa
-aaaaaaaaa
-aa
-aaaabbcd
bbccsdeqwww
aa
dmitrykochev$ ./lab2 <test1.txt
OK
OK
OK
OK
OK
OK
OK
NoSuchWord
OK: 7
NoSuchWord
OK: 6
OK
NoSuchWord
OK
OK
NoSuchWord
OK: 11
NoSuchWord
```

4 Тест производительности

Тесты производительности представляют из себя следующее: одни и те же данные будем вставлять, удалять и искать как в PATRICIA, так и в бинарном дереве поиска. Тестов будет три, тест характеризуется тремя числами: количество строк на вставку, поиск, и удаление из дерева. Первый тест будет содержать $5 * 10^4$ строк на вставку, $5 * 10^4$ на поиск и 10^4 на удаление. Второй 10^5 , 10^5 и $5 * 10^4$ соответственно. Третий 10^6 , 10^6 и $5 * 10^5$ соответственно. Длина строки не более 256 элементов.

```
dmitrykochev$ ./benchmark
insert to patricia: 1.4680
insert to bin tree: 1.4930
search in patricia: 1.1300
search in bin tree: 1.2480
delete in patricia: 0.2590
delete in bin tree: 0.2490
dmitrykochev$ ./benchmark
insert to patricia: 2.8980
insert to bin tree: 2.9380
search in patricia: 2.4210
search in bin tree: 2.5560
delete in patricia: 1.2850
delete in bin tree: 1.2910
dmitrykochev$ ./benchmark
insert to patricia: 31.3310
insert to bin tree: 33.6900
search in patricia: 25.4410
search in bin tree: 28.3910
delete in patricia: 13.2860
delete in bin tree: 14.5460
```

Как можно заметить, при увеличении числа строк PATRICIA становится заметно быстрее бинарного дерева.

Действительно, как говорилось выше, поиск в PATRICIA выполняется за $\Theta(h)$, в то же время в бинарном дереве поиска за $\Theta(h * m)$, где m - максимальная длина строки. Для вставки ситуация аналогична. Удаление у патриции работает за $\Theta(h)$, а в дереве, из-за использования строк вместо чисел, сложность будет равна $\Theta(h * m)$. Мы видим, что PATRICIA работает бинарного дерева даже с маленькой длинной строки. Это соответствует расчетам.

5 Профилирование и поиск утечек памяти

Gprof помогает отследить, какая из функций в процессе выполнения программы выполняется дольше всех.

Диагностика проводилась на тесте из 300000 строк на вставку, поиск и удаление:

```
dmitrykochev$ gcc -pg -o profile_laba2 laba2.c patricia.c
dmitrykochev$ ./profile_laba2 <tests.txt >out.txt
dmitrykochev$ gprof profile_laba2 >outgprof.txt
dmitrykochev$ head -n 17 outgprof.txt
Flat profile:
```

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self us/call	total us/call	name
44.90	0.44	0.44	299999	1.47	1.47	search
26.02	0.69	0.26	300000	0.85	1.47	toLowerCase
18.88	0.88	0.18	150000000	0.00	0.00	toLowerCase
3.06	0.91	0.03	99999	0.30	2.07	insertToPatricia
3.06	0.94	0.03	99523	0.30	0.30	searchNodeForInsert
2.04	0.96	0.02	57060	0.35	0.35	searchBackLinkToNode
1.02	0.97	0.01	100000	0.10	1.77	removeNode
1.02	0.98	0.01				_init
0.00	0.98	0.00	63036	0.00	0.00	searchDirectLinkToNode
0.00	0.98	0.00	1	0.00	0.00	createPatricia
0.00	0.98	0.00	1	0.00	0.00	deletePatricia

Видно, что больше всего времени программа тратит на функцию поиска. Это логично, ведь как и во вставке, так и в удалении используется функция поиска. Также много времени занимают функции, работающие с ключом. В моей программе каждая строка, подающаяся на вход, прогоняется через алгоритм, делающий каждый символ в ней нижнего регистра.

Valgrind позволяет увидеть, сколько было выделено и освобождено байт памяти в процессе выполнения программы, а также есть ли утечки памяти и в каких именно функциях.

```
dmitrykochev$ valgrind --leak-check=full -s ./laba2
==15310== Memcheck, a memory error detector
==15310== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==15310== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==15310== Command: ./laba2
```



```

==15310==
+ a 1
OK
==15310==
==15310== HEAP SUMMARY:
==15310==      in use at exit: 0 bytes in 0 blocks
==15310==    total heap usage: 4 allocs,4 frees,2,090 bytes allocated
==15310==
==15310== All heap blocks were freed --no leaks are possible
==15310==
==15310== Use --track-origins=yes to see where uninitialised values come from
==15310== ERROR SUMMARY: 519 errors from 2 contexts (suppressed: 0 from 0)
==15310== 498 errors in context 2 of 2:
==15310== Conditional jump or move depends on uninitialised value(s)
==15310==      at 0x10A704: toLower (in /mnt/c/Users/koche/mai/da/lab2_C/contest)
==15310==      by 0x10A75C: toLowercase (in /mnt/c/Users/koche/mai/da/lab2_C/contest)
==15310==      by 0x10ACB4: main (in /mnt/c/Users/koche/mai/da/lab2_C/contest)
==15310==
==15310== ERROR SUMMARY: 519 errors from 2 contexts (suppressed: 0 from 0)

```

Это сокращенный мной вывод данной утилиты. Видно, что после добавления всего одного ключа происходят 519 ошибок, но память очищается полностью. Также благодаря valgrind мы понимаем, что ошибка была в функции toLowercase. После исправления опять запустим проверку на большом тесте:

```

dmitrykochev$ valgrind --leak-check=full -s ./laba2 <tests.txt >out.txt
==21047== Memcheck,a memory error detector
==21047== Copyright (C) 2002-2017,and GNU GPL'd,by Julian Seward et al.
==21047== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==21047== Command: ./laba2
==21047==
==21047== HEAP SUMMARY:
==21047==      in use at exit: 0 bytes in 0 blocks
==21047==    total heap usage: 227,580 allocs,227,580 frees,20,624,712 bytes
allocated
==21047==
==21047== All heap blocks were freed --no leaks are possible
==21047==
==21047== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Утилита показывает, что все выделенные блоки памяти были освобождены и утечек памяти нет. Ошибок также нет.

Также после исправление этой ошибки функция toLowercase стала работать быстрее и теперь занимает меньше времени:

```
dmitrykochev$ gcc -pg -o profile_laba2 laba2.c patricia.c
dmitrykochev$ ./profile_laba2 <tests.txt >out.txt
dmitrykochev$ gprof profile_laba2 >outgprof.txt
dmitrykochev$ head -n 17 outgprof.txt
Flat profile:
```

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self us/call	total us/call	name
60.53	0.34	0.34	299999	1.15	1.15	search
9.65	0.40	0.06	300000	0.18	0.33	toLowerCase
8.77	0.45	0.05	99999	0.50	1.95	insertToPatricia
7.89	0.49	0.04	38520522	0.00	0.00	toLower
5.26	0.53	0.03	99523	0.30	0.30	searchNodeForInsert
1.75	0.54	0.01	100000	0.10	1.35	removeNode
1.75	0.55	0.01	57060	0.18	0.18	searchBackLinkToNode
1.75	0.56	0.01				_init
0.00	0.57	0.00	63036	0.00	0.00	searchDirectLinkToNode
0.00	0.57	0.00	1	0.00	0.00	createPatricia
0.00	0.57	0.00	1	0.00	0.00	deletePatricia

Отныне код оптимизирован и работает быстрее.

6 Дневник выполнения лабораторной работы

1. 24.04.2024 Первая реализация дерева Patricia.
2. 25.04.2024 Первая реализации требуемой задачи. Решение оказывается медленным.
3. 01.05.2024 Долгая работа над программой дает свои плоды. Исправление ошибок в загрузке в файл и выгрузки из него помогают дойти до финальных тестов. Там тестирующая система не принимает задачу из-за большого количества используемой памяти.
4. 02.05.2024 Ошибка в количестве используемой памяти решается изменением способа хранения ключа в дереве. Сначала я сразу хранил бинарное представление в памяти компьютера, но после исправления храню изначальную строку, а нужный бит высчитываю при его надобности. Это оказалось верным решением.
5. 08.04.2024 Произвел профилирование программы с помощью Gprof.
6. 08.04.2024 Использовал утилиту Valgrind для поиска утечек памяти. Все ошибки были исправлены. Также было ускорено время работы программы.

7 Выводы

Выполнив первую лабораторную работу по курсу «Дискретный анализ», я научился писать и тестировать структуру данных PATRICIA. Безусловно, это дало мне ценный опыт в написании структуры данных своими руками с нуля. Во время выполнения я сталкивался с огромным количеством проблем. В итоге я был удивлен, какая PATRICIA удобная, быстрая и изящная структура данных, позволяющая работать со строками как с числами. После данной лабораторной работы я убедился, что могу реализовывать даже такие сложные деревья, это придает мне уверенности в себе. Также я научился профилировать свой код и находить утечки памяти в нем. Благодаря этому мне удалось ускорить свою программу. Это невероятно ценные и важные знания. Они очень помогут мне в будущей разработке сложных задач и программ и приблизили меня к настоящей работе программистом.

Список литературы

- [1] Dinesh P. Mehta, Sartaj Sahni, *Handbook of DATA STRUCTURES and APPLICATIONS*.